definitely reduces the usability of the script. To be useful, we should be passing the application ID as a parameter:

```
#!/bin/bash
yarn –log -applicationId ${1} | less
```

We need to execute the script as:

```
./show_log.sh 123
```

The script will execute the *yarn* command, fetch the log for the application and allow us to view it.

What if we want to redirect the output to a file? Not a problem. Instead of sending the output to *less*, we can redirect it to a file:

```
#!/bin/bash
ls –l ${1} > ${2}
view ${2}
```

To run the script, we have to provide two parameters, and the command becomes:

```
./my_file.sh /tmp /tmp/listing.txt
```

When executed, $1 will bind to */tmp* and $2 will bind to */tmp/listing.txt*. For the shell, the parameters are named from one to nine. This does not mean we cannot pass more than nine parameters to a script. We can, but that is the topic of another article. You will note that I have mentioned the parameters as ${1} and ${2} instead of $1 and $2. It is a good practice to enclose the name of the parameter in curly brackets as it allows us to unambiguously combine the parameters as part of a longer variable. For example, we can ask the user to provide file name as a parameter and then use that to form a larger file name. As an example, we can take $1 as the parameter and create a new file name as *${1}_student_names.txt*.

Making the script robust

What if the user forgets to provide parameters? The shell allows us to check for such conditions. We modify the script as below:

```
#!/bin/bash
if [ -z "${2}" ]; then
    echo "file name not provided"
    exit 1
fi
if [ -z "${1}" ]; then
    echo "directory name not provided"
    exit 1
fi
DIR_NAME=${1}
FILE_NAME=${2}
ls -l ${DIR_NAME} > /tmp/${FILE_NAME}
view /tmp/${FILE_NAME}
```

In this program, we check if the proper parameters are passed. We exit the script if parameters are not passed. You will note that I am checking the parameters in reverse order. If we check for the presence of the first parameter before checking the presence of the second parameter, the script will pass to the next step if only one parameter is passed. While the presence of parameters can be checked in ascending order, I recently realised that it might be better to check from nine to one, as we can provide proper error messages. You will also note that the parameters have been assigned to variables. The parameters one to nine are positional parameters. Assigning positional parameters to named parameters makes it easy to debug the script in case of issues.

Automating backup

Another task that I automated was that of taking a backup. During development, in the initial days, we did not have a version control system in place. But we needed to have a mechanism to take regular backups. So the best method was to write a shell script that, when executed, copied all the code files into a separate directory, zipped them and then uploaded them to HDFS, using the date and time as the suffix. I know that this method is not as clean as having a version control system, as we store complete files and finding differences still needs the use of a program like *diff*; however, it is better than nothing. While we did not end up deleting the code files, the team did end up deleting the *bin* directory where the helper scripts were stored!!! And for this directory I did not have a backup. I had no choice but to re-create all the scripts.

Once the source code control system was in place, I easily extended the backup script to upload the files to the version control system in addition to the previous method uploading to HDFS.

Summing up

These days, programming languages like Python, Spark, Scala, and Java are in vogue as they are used to develop applications related to artificial intelligence and machine learning. While these languages are far more powerful when compared to shells, the 'humble' shell provides a ready platform that allows us to create helper scripts that ease our day-to-day tasks. The shell is quite powerful, more so because we can combine the powers of all the applications installed on the OS. As I found out in my project, even after many